

BHOOMISCAN | YOUR AIVOCATE.

AI-Powered Property Due Diligence Platform

A detailed project report covering abstract, introduction, methodology, requirements, design, implementation, testing, results, conclusion, and references.

Submitted in partial fulfilment of the academic project

Product	BhoomiScan - Your Alvocate.
Workspace	BhoomiScanV18
Version	0.1.0
Stack	React 19, FastAPI, PostgreSQL 16, Sarvam Document AI
Date	September 2026

Disclaimer: This system is an AI-assisted document review. It is not a lawyer, not a government portal, and not a legal guarantee of a clean title.

Table of Contents

Chapter 1 Abstract	3
Chapter 2 Introduction	4
Chapter 3 Methodology and Proposed System	6
Chapter 4 System Requirements Specification	9
Chapter 5 System Design	12
Chapter 6 Implementation	15
Chapter 7 System Testing	19
Chapter 8 Results	22
Chapter 9 Conclusion and Future Enhancement	24
Chapter 10 References	25

List of figures in this report includes architecture, data flow, entity relationships, and the analysis pipeline. Tables include requirements, API endpoints, risk rules, and test cases.

Chapter 1 Abstract

Property purchase in India, especially in Karnataka, depends on a bundle of scanned papers: sale deeds, encumbrance certificates, khata extracts, survey sketches, building approvals, and court papers. Many of these documents are in Kannada, poorly scanned, and written in legal language that a first-time buyer cannot reliably interpret. Missing papers are often treated as the absence of a problem. Cross-document mismatches, such as a sale deed quoting survey number 45/3 while a survey sketch quotes 45/4, are easy to miss when files are checked one at a time. Commercial due-diligence services exist, but they are costly, often English-only, or overclaim that a property is legally clear.

BhoomiScan (tagline: Your Alvocate.) is a student-scale full-stack web application that helps a buyer check residential property papers before purchase. A logged-in user creates a residential property case, uploads PDF or image files, and runs analysis. Sarvam Document AI performs optical character recognition and Kannada-to-English translation. A deterministic Python risk engine, not a large language model, then classifies documents, extracts structured fields, evaluates five risk categories, compares values across files, scores risk, and produces a versioned report. The report is hashed with SHA-256 and appended to a local GENESIS hash chain so later tampering can be detected.

The academic contribution is a hybrid, explainable pipeline: Indic Document AI is used only to read text, while Python rules decide DETECTED, NO_ISSUE_FOUND, or NOT_VERIFIED. Absence of evidence is never treated as a clean title. The implemented system covers JWT authentication, document vault, analysis, case-level recalculation, comparison, coverage versus risk score, PDF export, and integrity verification. Automated pytest coverage comprises 39 cases across three phases, with Sarvam mocked so tests remain deterministic. The system does not scrape government portals, process live payments, train custom models, or claim legal certainty.

Title	BhoomiScan - AI-Powered Property Due Diligence
Domain	Residential property document review
Languages handled	English and Kannada (OCR + translation)
AI role	Read and extract; Python decides risk
Integrity	SHA-256 + local GENESIS ledger
Tests	39 pytest cases (13 + 7 + 19)

Chapter 2 Introduction

2.1 Overview

Buying a home is one of the largest financial decisions a family makes. In the Indian registration and revenue system, title is reconstructed from documents rather than from a single conclusive online register that a buyer can trust without reading papers. BhoomiScan is a local web product that gives a buyer a structured first pass over those papers. It stores files in a case vault, reads them, highlights potential issues with evidence snippets, recommends missing documents, and writes a tamper-evident report.

The product is intentionally modest in legal claims. Allowed language includes phrases such as Potential issue detected, Based on the documents provided, Could not be verified, and Further verification is recommended. Forbidden language includes Property is completely safe, legally clear, or guaranteed clean. The on-screen and stored disclaimer states that the report does not replace a qualified lawyer, surveyor, or government authority.

2.2 Problem statement

Buyers receive a folder of scans. Some pages are in Kannada. Some are photos of bound registers. Risk is distributed across five practical questions: who owns the property, whether a court case or stay exists, whether a bank charge is open, whether construction was approved, and whether survey, khata, area, and boundary details match across papers. Manual review is slow and inconsistent. Spreadsheet checklists do not OCR Kannada. Generic chatbots may invent a clean title when a document is silent. The problem addressed by this project is:

Can a student-scale full-stack system combine Indic Document AI with deterministic rules to present DETECTED / NOT_VERIFIED / NO_ISSUE_FOUND with evidence, without faking AI, payments, OAuth, or a public blockchain?

2.3 Motivation

- Kannada sale deeds and court extracts are common in Karnataka residential deals.
- A missing encumbrance certificate must remain Mortgage NOT_VERIFIED, not a green tick.
- Survey mismatches between a deed and a sketch are a frequent source of later dispute.
- Students need an end-to-end artefact that is explainable in a viva: upload, OCR, rules, hash.
- Existing tools either stay fully manual or hide scoring inside an opaque LLM prompt.

2.4 Objectives

2.4.1 Primary objective

Build an end-to-end local web system: sign up and log in, create a residential case, upload documents, run AI analysis, recommend more papers, compare values, recalculate case-level risk, finalize a versioned report, hash it, write a local ledger, and verify integrity.

2.4.2 Specific objectives

1. Provide username/password authentication with bcrypt hashes and JWT access tokens.
2. Restrict live analysis to RESIDENTIAL property types; gate other domains to a subscription UI.
3. Accept PDF, PNG, and JPEG uploads up to 10 MB with magic-byte validation.

4. Call Sarvam Document AI to digitise pages and translate Kannada text in 1800-character chunks.
5. Classify document type by keywords with a confidence floor of 0.45.
6. Evaluate five risk categories using three statuses and four severity levels.
7. Never treat missing evidence as NO_ISSUE_FOUND.
8. Compare extracted fields across documents with MATCH / MISMATCH / NOT_AVAILABLE.
9. Keep verification coverage percent independent of the numeric risk score.
10. Store SHA-256 of canonical report JSON and chain records from GENESIS.
11. Protect case ownership with 401, 403, and 404 responses.
12. Cover the pipeline with pytest using an in-memory SQLite database and mocked Sarvam.

2.4.3 Non-objectives

- Public blockchain, wallets, mining, smart contracts, or cryptocurrency.
- Live UPI, card, or subscription billing. Get Subscription is a user-interface placeholder.
- Real Google, X, or phone OAuth. Those buttons are demo-only.
- Scraping Kaveri, Bhoomi, BBMP, or other government portals.
- Custom machine-learning training, microservices, or Kubernetes.
- Agricultural, commercial, or industrial analysis engines.
- Any statement that the property is legally clear.

2.5 Scope

Phase 1 - Foundation

Landing page, signup, login, JWT, dashboard, property domain, residential types, property case, document upload, protected routes, and a dark royal user interface. Agricultural, commercial, and industrial domains show Get Subscription. Thirteen automated tests live in tests/test_phase1.py.

Phase 2 - Document analysis

Sarvam digitise, translate, and extract; classifier; merge extraction with a source-text veto so an AI JSON object cannot hide a stay order present in OCR text; evidence snippets; Python risk engine; recommendations; analysis screens. Seven tests live in tests/test_phase2.py.

Phase 3 - Case-level due diligence

Multi-document vault, delete, recalculate without re-calling Sarvam, comparison table, coverage, risk score, finalize dialog, versioned reports, SHA-256, ledger, verify, and PDF download. Nineteen tests live in tests/test_phase3.py.

2.6 Organization of the report

Chapter 1 states the abstract. Chapter 2 introduces the problem and objectives. Chapter 3 describes methodology and the proposed system. Chapter 4 records software requirements. Chapter 5 presents architecture and data design. Chapter 6 explains implementation. Chapter 7 documents testing. Chapter 8 reports results and observations. Chapter 9 concludes and lists future work. Chapter 10 lists references.

Chapter 3 Methodology and Proposed System

3.1 Existing system

Today a buyer, a family member, or a junior advocate reads papers page by page. Checklists may exist in Word or Excel. Translation of Kannada pages is informal. There is no automatic cross-check of survey numbers. There is no stored evidence trail that a later reader can verify. If a file is edited after a verbal all-clear, nothing in the folder proves what was reviewed. Paid opinion letters exist, but they are not a self-serve student tool and they do not expose a testable rule engine.

3.2 Limitations of the existing approach

- Language barrier: Kannada OCR and translation are not part of a typical checklist.
- Absence bias: no EC is read as no loan.
- No comparison: two files are not normalized and compared field by field.
- No integrity: a PDF can be swapped after a review.
- Opaque AI: a chatbot answer cannot be unit-tested as MATCH or MISMATCH.

3.3 Proposed system

BhoomiScan is a browser single-page application talking to a FastAPI backend. PostgreSQL stores users, cases, document metadata, OCR text, extracted JSON, findings, comparisons, reports, and ledger rows. Original bytes stay on disk under storage/documents/{case_id}/. Sarvam is invoked only when the user clicks Analyze. Recalculate reuses stored text so the buyer can add an encumbrance certificate later without paying for OCR again.

3.4 Hybrid AI plus rules

Sarvam Document AI is treated as an input device. It digitises pages, optionally extracts a JSON object matching a declared schema, and translates Kannada. Python then classifies the document, merges AI fields with regular-expression fallbacks, applies a source-text veto for litigation language, evaluates risks, compares documents, scores, hashes, and writes the ledger. An LLM does not decide risk status or MATCH versus MISMATCH. This split is the core methodological choice: reading can be probabilistic; verdicts must be deterministic and testable.

3.5 Three-state verification

Each of the five categories receives exactly one status:

- DETECTED: explicit adverse evidence exists (stay order, pending case, incomplete names, unauthorized construction, uncleared mortgage, or a high-severity mismatch).
- NO_ISSUE_FOUND: a relevant document explicitly clears the category (for example an encumbrance certificate stating nil encumbrance).
- NOT_VERIFIED: supporting papers are missing or silent. This is the default. Wrong rule: no EC means no mortgage. Right rule: no EC means Mortgage NOT_VERIFIED.

This three-state model is more honest than a binary safe/unsafe flag. It forces the interface to show unverified work instead of a false green tick.

3.6 Evidence-first findings

A finding stores a summary plus evidence rows: source snippet, optional page, document type, extracted field, and confidence. If seller and buyer names are missing but survey 45/3 appears in the OCR, the ownership finding can still attach the survey snippet and state that names were not found. The user can open an evidence panel instead of trusting a score alone.

3.7 Document analysis versus case recalculation

Document analysis operates on one file and may call Sarvam. Case recalculation loads every analyzed file in the vault, merges category statuses with DETECTED winning over NO_ISSUE_FOUND winning over NOT_VERIFIED, overlays MISMATCH results, recomputes coverage and score, and refreshes recommendations. Recalculate does not re-OCR.

3.8 Coverage versus risk score

Two numbers are shown on due diligence so a high score is not confused with completeness, and a high coverage is not confused with safety.

- Risk score: points only from DETECTED findings. HIGH adds 70, MEDIUM adds 15, LOW adds 10, cap 100. NOT_VERIFIED adds zero.
- Overall level: HIGH if any DETECTED HIGH or score ≥ 40 ; MEDIUM if any DETECTED MEDIUM or score ≥ 10 ; otherwise LOW.
- Coverage: Verified = 1.0 (DETECTED or NO_ISSUE_FOUND), Partially Verified = 0.5 (NOT_VERIFIED but a related paper was uploaded), Not Verified = 0.0. Percent = $\text{round}(100 * \text{sum} / 5)$.

3.9 Tamper evidence

When the user finalizes, the backend builds a canonical JSON object with sorted keys, computes `report_hash` = SHA-256(canonical JSON), and writes a ledger row. `record_hash` = SHA-256 of `report_id`, `report_hash`, `previous_hash`, and timestamp. The first `previous_hash` is the literal GENESIS. A later version links to the previous `record_hash`. Verify recomputes hashes; it does not blindly trust the stored value. Chain order uses report version, not wall-clock timestamps, to avoid false TAMPERED results from time zones.

3.10 Development methodology

Work was delivered in three incremental phases with automated tests at each gate. Phase 1 proved auth and upload. Phase 2 proved analysis with mocked OCR. Phase 3 proved comparison, scoring, and the ledger. This is iterative delivery with a stable REST contract rather than a single waterfall dump. Frontend and backend stay as one Vite app and one FastAPI process. No extra microservice was introduced.

3.11 Proposed user journey

1. Landing -> Sign up with username and matching password -> Login (JWT stored in localStorage key `bhoomiscan.token`).
2. Dashboard -> Start Property Analysis -> choose RESIDENTIAL -> choose vacant land, house, flat, villa, or plot.
3. Create property case -> document vault -> upload PDF/PNG/JPG -> Analyze (up to 180 seconds).
4. Review extracted fields, five risk cards, evidence, and recommended papers.
5. Upload more files (EC, sketch, approval) -> Analyze each -> Recalculate.

6. Due diligence shows overall risk, score /100, coverage %, MATCH/MISMATCH, unverified calls to action.
7. Finalize with the warning that some checks may remain unverified. Missing documents do not block the report.
8. Open report: VALID or TAMPERED, SHA-256, record number, Verify Integrity, Download PDF.

Chapter 4 System Requirements Specification

4.1 Product perspective

BhoomiScan is a standalone local application. The browser talks to `http://localhost:8000`. The Vite development server serves `http://localhost:5173`. PostgreSQL listens on 5432 via Docker Compose (database, user, and password all set to `bhoomiscan`). Live OCR requires `SARVAM_API_KEY` in the project-root `.env` file. Secrets never ship to the React bundle; only `VITE_API_URL` is exposed to the frontend.

4.2 Functional requirements

ID	Requirement	Priority
FR-01	Sign up with unique username and matching passwords	Must
FR-02	Login and return JWT; <code>/auth/me</code> for current user	Must
FR-03	Create residential property case for the owner	Must
FR-04	Reject non-residential case creation with 400	Must
FR-05	Upload PDF/PNG/JPG; reject other types and oversize	Must
FR-06	List and delete own documents	Must
FR-07	Analyze: OCR, translate Kannada, extract, risk	Must
FR-08	Show evidence snippets per finding	Must
FR-09	Recommend missing document types	Must
FR-10	Recalculate case using stored analyses	Must
FR-11	Compare fields: MATCH / MISMATCH / NOT_AVAILABLE	Must
FR-12	Compute coverage % and risk score separately	Must
FR-13	Finalize versioned report; never silent-edit	Must
FR-14	SHA-256 + GENESIS ledger; verify VALID/TAMPERED	Must
FR-15	Download due-diligence PDF	Must
FR-16	Ownership 401/403/404 on all case routes	Must
FR-17	Subscription UI for non-residential domains	Should
FR-18	Demo social login buttons (non-functional)	Could

4.3 Non-functional requirements

ID	Quality	Specification
NFR-01	Security	bcrypt passwords; JWT HS256; CORS allow-list
NFR-02	Integrity	NUL bytes stripped from OCR before PostgreSQL
NFR-03	Safety of files	Uploads stored, never executed
NFR-04	Usability	Simple English; dark navy, gold, ivory UI
NFR-05	Performance	Analyze timeout 180s; Sarvam poll 3s, max 120s
NFR-06	Testability	SQLite in-memory tests; Sarvam mocked

ID	Quality	Specification
NFR-07	Maintainability	Routers + services + SQLAlchemy models
NFR-08	Portability	Windows/macOS/Linux with Docker Postgres
NFR-09	Legal tone	Disclaimer on reports; no clear-title claim
NFR-10	Reliability	Recalculate works if Sarvam is later down

4.4 Hardware requirements

Processor	Any modern dual-core CPU suitable for Node and Python
Memory	8 GB RAM recommended (Docker + Node + Python)
Disk	About 2 GB for images, venv, node_modules, and uploads
Display	1280 x 720 or larger for the royal-themed SPA
Network	Internet required only for Sarvam OCR/translate

4.5 Software requirements

OS	Windows 10/11 (developed on Windows 10.0.26200)
Python	3.11+ (also ran on 3.14.7)
Node.js	18+ (development used 22.14.0)
Database	PostgreSQL 16 Alpine via Docker Compose
Runtime DB URL	postgresql+psycopg://bhoomiscan:bhoomiscan@localhost:5432/bhoomiscan
Test DB	SQLite in-memory with StaticPool
Backend packages	FastAPI, Uvicorn, SQLAlchemy 2, PyJWT, bcrypt, fpdf2, pytest
AI SDK	sarvamai plus requests for download URLs
Frontend	React 19, Vite 6, React Router 7, Tailwind 3, Axios, Framer Motion

4.6 Actors and use cases

Actors

- Guest: sees landing, signup, and login.
- Authenticated buyer: owns cases, uploads, analyzes, finalizes.
- System / Sarvam: external OCR and translation provider.
- Database and disk: persist metadata and original files.

Primary use cases

Use case	Actor	Result
UC-01 Register	Guest	User row; password hash stored
UC-02 Login	Guest	JWT for subsequent calls
UC-03 Create case	Buyer	RESIDENTIAL case ACTIVE
UC-04 Upload paper	Buyer	File on disk + documents row
UC-05 Analyze	Buyer + Sarvam	OCR, risks, evidence

Use case	Actor	Result
UC-06 Recalculate	Buyer	Merged case view, no OCR
UC-07 Finalize	Buyer	New report version + ledger
UC-08 Verify	Buyer	VALID or TAMPERED
UC-09 Download PDF	Buyer	fpdf2 risk report
UC-10 Delete file	Buyer	Own document removed

4.7 External interface requirements

Human interface: React pages at / , /login, /signup, /dashboard, /property-domain, /property-type, /property-case/:id/upload, analysis, due-diligence, and reports. Software interface: REST JSON with Authorization Bearer tokens. Communication: HTTP on loopback. Sarvam is reached with the official SDK using the server-side API key.

4.8 Constraints, assumptions, and dependencies

- Assumption: the buyer uploads genuine scans of the property they intend to buy.
- Assumption: Kannada Unicode in OCR is a sufficient language detector (8+ Kannada letters or more Kannada than Latin).
- Constraint: OCR quality depends on scan contrast and Sarvam availability.
- Constraint: only kn-IN and en-IN are first-class languages in the pipeline.
- Dependency: Docker engine must be running for PostgreSQL.
- Dependency: SARVAM_API_KEY for live Analyze; tests do not need it.
- Constraint: JWT_SECRET must remain on the server.

4.9 Allowed document types

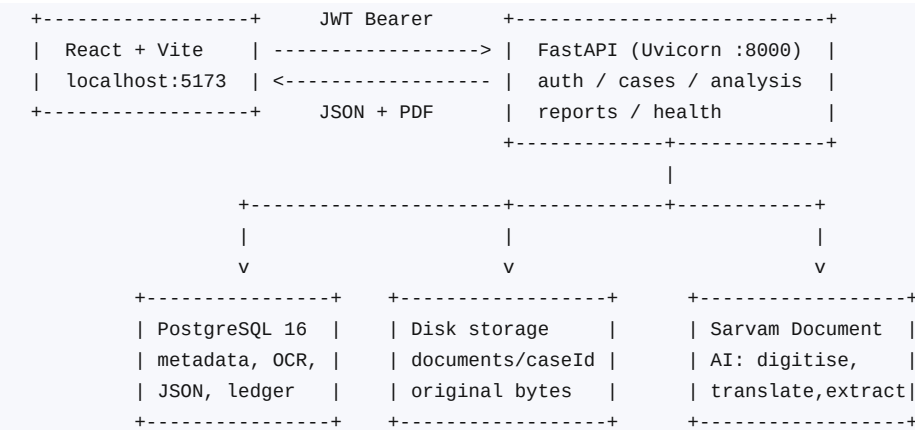
The classifier and recommendation engine know the following types: Sale Deed, Parent Deed, Encumbrance Certificate, Khata / Property Register, Mutation, Court Case Document, Court Order, Legal Notice, Seller Affidavit, Bank Mortgage Document, Loan Closure NOC, Release Deed, Bank NOC, Building Plan, Building Plan Approval, Commencement Certificate, Occupancy / Completion Certificate, Land Conversion, Survey Sketch, Property Tax Receipt, Site Layout Plan, and Other. Confidence below 0.45 maps to Other.

Chapter 5 System Design

5.1 Architectural style

The system is a layered client-server application. Layer 1 is the React SPA with protected routes and a royal visual theme. Layer 2 is REST routers with JWT and ownership checks. Layer 3 is domain services (auth, storage, Sarvam, classifier, extraction, risk, comparison, scoring, hashing, ledger, PDF). Layer 4 is SQLAlchemy models on PostgreSQL. Layer 5 is the Sarvam network I/O boundary. This split keeps AI reading away from rule decisions so both can be tested.

5.2 System architecture



5.3 Context (Level-0) data flow

The buyer provides credentials and document files. BhoomiScan returns risk findings, recommendations, a score, coverage, and a hashed report. Sarvam receives file bytes during Analyze and returns text or JSON. PostgreSQL and disk are data stores, not external actors.

5.4 Level-1 data flow

```

[Buyer] --credentials--> (Auth) --JWT--> [Buyer]
[Buyer] --file bytes--> (Storage) --path--> (Analyze)
(Analyze) --bytes--> [Sarvam] --OCR/JSON--> (Analyze)
(Analyze) --text--> (Classifier / Extract / Risk / Evidence)
(Risk) --findings--> (Recommend)
[Buyer] --recalculate--> (Case merge + Compare + Score)
[Buyer] --finalize--> (Report JSON) --SHA-256--> (Ledger)
[Buyer] --verify--> (Hash recompute) --VALID/TAMPERED--> [Buyer]

```

5.5 Sequence: Analyze one document

1. Frontend POST /property-cases/{id}/documents/{doc}/analyze with JWT.
2. Router loads the document, confirms the case belongs to the user, sets status PROCESSING.
3. digitise_document sends bytes to Sarvam with language kn-IN, polls until completed, reads page blocks, falls back to ZIP download URL.
4. looks_like_kannada inspects Unicode range U+0C80 to U+0CFF. If Kannada, translate_text chunks at 1800 characters to en-IN.

5. `classify_document` scores filename plus working text against type keywords.
6. `extract_fields` asks Sarvam for schema JSON; on failure uses {}.
7. `merge_extraction` combines AI JSON, regex fallbacks, and source-text veto for stay/case/restriction.
8. `evaluate_risks` and `build_recommendations` persist findings, evidence, and recommendations.
9. Status ANALYZED or FAILED. Frontend polls or navigates to the analysis page.

5.6 Entity-relationship design

UUID primary keys are used throughout. Cascades delete child rows when a case or analysis is removed. Document deletion sets comparison document foreign keys to NULL where needed.

Entity	Key attributes	Relates to
User	id, username, password_hash	1..* PropertyCase
PropertyCase	domain, property_type, status	User, Documents, Analyses
Document	path, type, OCR, extracted JSON	Case, Analyses
Analysis	status, pipeline_steps, JSON	Case, Document, Findings
RiskFinding	category, status, level	Analysis, Evidence
EvidenceItem	source_text, page, field	Finding, Document
Recommendation	category, document_type	Analysis
DocumentComparison	field, values, result	Case, two Documents
FinalReport	version, hash, score	Case, LedgerRecord
LedgerRecord	previous_hash, record_hash	FinalReport (1:1)

Startup calls `Base.metadata.create_all` and `ensure_schema` so Phase 2 columns such as `detected_language` can be added on an existing documents table without dropping user data.

5.7 Module design

Backend routers

- `auth.py`: signup, login, me.
- `property_cases.py`: cases, upload, list, delete.
- `analysis.py`: analyze, get analysis, recommendations, evidence, recalculate, due-diligence, comparisons.
- `reports.py`: finalize, list, get, verify, PDF.

Backend services

- `sarvam_service.py`: live digitise, translate, extract.
- `analysis_service.py`: one-document pipeline.
- `case_analysis_service.py`: merge all analyzed files.
- `risk_engine.py` and `recommendation_engine.py`: category rules.
- `comparison_service.py`: normalized pairwise compare.
- `scoring_service.py`: points, overall level, coverage.
- `hashing.py` and `ledger_service.py`: SHA-256 chain.
- `report_service.py` and `report_pdf.py`: versioned JSON and PDF.
- `storage.py`: safe filenames and magic-byte checks.

Frontend pages

- Public: Landing, Login, Signup.
- Protected: Home, AI helper chat (demo, not Sarvam), Profile, Settings, Contact, Subscription.

- Flow: PropertyDomain, PropertyType, DocumentUpload vault, DocumentAnalysis, DueDiligence, FinalReport.
- Shared: ProtectedRoute, FlowStepper, EvidencePanel, ComparisonTable, RiskCategoryCard.

5.8 User-interface design

The visual language is dark navy backgrounds, gold accents, ivory type, and rounded cards. Motion uses Framer Motion page transitions. Copy is kept in simple English so a non-lawyer can follow the five checks. The stepper shows Property kind, Home type, Document vault, and Due diligence. Logged-in navigation shows Home, AI, profile, and Sign Out. Dashboard does not invent fake statistics.

5.9 Security design

- Passwords hashed with bcrypt; hashes never returned in JSON.
- Protected endpoints require Authorization: Bearer <token>; else 401.
- Another user's case or document returns 403; unknown id returns 404.
- Uploads checked by extension, declared MIME, and file signatures (%PDF, PNG, JPEG).
- CORS origins limited to http://localhost:5173 and http://127.0.0.1:5173.
- Frontend never computes a trusted risk score; the backend always recomputes.
- API keys remain in server .env only.

5.10 Report content design

report_content JSON includes property info, document list, risk summary, overall level, coverage, detected issues, evidence, mismatches, unverified categories, recommendations, score reasons, and the legal disclaimer. A new finalize always increments version. Historical reports are immutable.

Chapter 6 Implementation

6.1 Technology stack

Python FastAPI was chosen for typed request models, automatic OpenAPI, and easy TestClient tests. SQLAlchemy 2 provides mapped models with UUID keys. React 19 with Vite gives a fast SPA. Tailwind expresses the royal theme without a heavy component library. fpdf2 writes Latin-1-safe PDFs. Docker Compose pins PostgreSQL 16 so classmates can start the database with one command.

6.2 Environment and configuration

Pydantic Settings reads the root .env: DATABASE_URL, JWT_SECRET, JWT_ALGORITHM (HS256), JWT_EXPIRE_MINUTES (1440), MAX_FILE_SIZE_MB (10), CORS_ORIGINS, SARVAM_API_KEY, SARVAM_POLL_SECONDS (3), SARVAM_MAX_WAIT_SECONDS (120). Frontend .env contains only VITE_API_URL.

6.3 Authentication implementation

Signup validates matching passwords and unique usernames (409 if taken). Login verifies bcrypt and returns access_token. deps.py decodes JWT and loads the user. Tokens expire after 24 hours by default. Google, X, and phone buttons are visual only.

6.4 Property case and upload implementation

POST /property-cases accepts domain and property_type. If domain is not RESIDENTIAL the API returns 400 directing the client to subscription. Allowed residential types are vacant land, independent house, apartment or flat, villa, and residential plot. Upload writes a sanitized filename under storage/documents/{property_case_id}/ and a documents row with status UPLOADED. Empty files and files over the limit are rejected.

6.5 Sarvam integration

sarvam_service.py constructs a SarvamAI client from the API key. Digitise submits (filename, bytes, mime), polls get_status until a terminal status in {completed, partially_completed, failed, rejected}, then prefers get_results page content and blocks. If results are empty it downloads a URL which may be a ZIP of markdown or HTML. Translate uses sarvam-translate:v1 from kn-IN to en-IN. Extract sends EXTRACTION_SCHEMA as JSON. Any extract failure is swallowed into {} so regex fallback can still run. Missing API key raises a clear SarvamError.

6.6 Extraction schema and merge

The schema groups property identifiers (survey, khata, area, village, hobli, taluk, district, state, type), ownership names, transaction amounts and registration numbers, litigation flags, mortgage flags, approval flags, and four boundaries. merge_extraction fills blanks from regular expressions on working text and original text. A source-text veto reasserts stay, pending case, or transfer restriction if those words exist even when the AI JSON omitted them. That rule is essential: the model must not be allowed to hide a court stay.

6.7 Classification

document_classifier.py scores known phrases in filename and text. A litigation-heavy sale deed that repeats O.S. No. may classify as Court Case Document; that is a known limitation, not a crash. looks_like_kannada counts Kannada versus Latin letters.

6.8 Risk engine

Ownership / Title

Missing seller or buyer names produce DETECTED at MEDIUM. Identifiers without supporting title documents remain NOT_VERIFIED. Title is never auto-cleared from a survey number alone.

Litigation

Pending case, stay, restriction, or a case number in source text produces DETECTED, typically HIGH. Explicit no pending litigation on a verifying document type may become NO_ISSUE_FOUND. Otherwise NOT_VERIFIED. The word lien is matched on word boundaries so alienation is not treated as a mortgage.

Mortgage / Encumbrance

An open charge without closure is DETECTED. An encumbrance certificate with nil / no / free from encumbrance can become NO_ISSUE_FOUND after recalculate. Silence remains NOT_VERIFIED.

Approval / Construction

Unauthorized construction or deviation is DETECTED. Approval-type documents plus present or occupancy certificate can become NO_ISSUE_FOUND. A sale deed alone is not building permission.

Property record / Boundary

Identifiers stay NOT_VERIFIED until cross-document comparison. A survey or boundary MISMATCH overlays DETECTED at HIGH or MEDIUM depending on field.

Case merge

When several documents speak, DETECTED wins. Else any NO_ISSUE_FOUND is kept. Else NOT_VERIFIED. Mismatches then overlay the merged set.

6.9 Comparison algorithm

comparison_service.py compares seller, buyer, owner, survey, khata, area, village, hobli, taluk, district, property type, north/south/east/west, registration number and date, and previous deed number. Normalization strips extra whitespace, punctuation, and prefixes such as Survey No. and Sy. No. Person names collapse spaces and case; different names are not merged. Area converts to square feet (square metres * 10.7639) with 2% tolerance. If fewer than two documents carry a value the result is NOT_AVAILABLE. Otherwise pairwise MATCH or MISMATCH with severity and a plain-language explanation.

6.10 Scoring implementation

scoring_service.py sums RISK_SCORE_POINTS only for DETECTED rows and caps at 100. score_reasons attach category labels for the Because section of the PDF. verification_coverage walks the five categories and related uploaded types from the recommendation map so Partial credit is possible when a related paper exists but the category is still NOT_VERIFIED.

6.11 Report, hash, and ledger implementation

finalize creates FinalReport with an incremented version, stores report_content, computes report_hash, and inserts LedgerRecord. Verify recomputes the content hash, recomputes the ledger material hash, and checks that previous_hash is GENESIS or the prior version's record_hash. A modified JSON yields TAMPERED. PDF export uses fpdf2 with the same disclaimer footer used in this academic report's spirit: not a legal guarantee.

6.12 Frontend implementation notes

Axios in services/api.js attaches the JWT and uses a long timeout for analyze. ProtectedRoute redirects guests to login. ConfirmDialog warns before finalize. AnalysisProgress lists uploaded, reading, extracting, checking, and complete. The in-app AI page is a demo helper chatbot, not the Sarvam analysis path, and must not be confused with document OCR.

6.13 API catalogue

Method	Path	Auth
POST	/auth/signup	No
POST	/auth/login	No
GET	/auth/me	JWT
POST	/property-cases	JWT
GET	/property-cases	JWT
GET	/property-cases/{id}	JWT
POST	/property-cases/{id}/documents	JWT
GET	/property-cases/{id}/documents	JWT
DELETE	.../documents/{doc}	JWT
POST	.../documents/{doc}/analyze	JWT
GET	.../documents/{doc}/analysis	JWT
GET	/property-cases/{id}/analysis	JWT
GET	.../{id}/recommendations	JWT
GET	.../{id}/evidence/{fid}	JWT
POST	.../{id}/recalculate	JWT
GET	.../{id}/due-diligence	JWT
GET	.../{id}/comparisons	JWT
POST	.../{id}/finalize	JWT
GET	.../{id}/reports	JWT
GET	.../reports/{rid}	JWT
GET	.../reports/{rid}/verify	JWT
GET	.../reports/{rid}/pdf	JWT
GET	/health	No

6.14 How to run the implementation

Start Docker Desktop, then from the project root run `docker compose up -d`. In backend, activate `.venv` and run `python -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000`. In frontend run `npm run dev`. Open `http://localhost:5173`. Health check: `GET http://127.0.0.1:8000/health`.

Chapter 7 System Testing

7.1 Test strategy

Testing follows the same three phases as delivery. Automated tests use pytest and FastAPI TestClient. Each test gets an isolated SQLite in-memory database. Sarvam HTTP calls are monkeypatched so CI and laptops without an API key still pass. Live UI testing of OCR requires SARVAM_API_KEY and a real Kannada PDF. Frontend production bundling is checked with npm run build.

7.2 Test environment

Harness	pytest + FastAPI TestClient
Automated DB	SQLite in-memory, StaticPool
Runtime DB	PostgreSQL 16 Docker
AI in pytest	Mocked digitise / translate / extract
AI in UI	Live Sarvam when key is set
Suites	test_phase1.py, test_phase2.py, test_phase3.py
Count	13 + 7 + 19 = 39 test functions

7.3 Phase 1 test cases (foundation)

ID	Function	Expected
TC-01	test_signup_success	201; username returned; no password
TC-02	test_duplicate_signup	409 Username already exists
TC-03	test_signup_password_mismatch	422 passwords must match
TC-04	test_login_success	JWT access_token
TC-05	test_invalid_login	401 invalid credentials
TC-06	test_protected_endpoint_requires_auth	401 without token
TC-07	test_protected_endpoint_with_token	200 with token
TC-08	test_property_case_creation	Residential case created
TC-09	test_property_case_rejects_non_residential	400
TC-10	test_document_upload	PDF stored; metadata saved
TC-11	test_unauthorized_case_access	403 other user's case
TC-12	test_missing_property_case	404
TC-13	test_unsupported_file	Reject .txt

7.4 Phase 2 test cases (analysis)

ID	Function	Expected
TC-14	test_analyze_requires_authentication	401
TC-15	test_analyze_uploaded_pdf	Mocked OCR pipeline completes

ID	Function	Expected
TC-16	test_user_cannot_analyze_another_users_document	403
TC-17	test_missing_document_returns_404	404
TC-18	test_litigation_detected_from_explicit_evidence	DETECTED
TC-19	test_empty_evidence_does_not_produce_no_issue_found	NOT_VERIFIED
TC-20	test_recommendation_engine_returns_missing_documents	Suggestions

TC-19 is the academic linchpin: empty evidence must not collapse into NO_ISSUE_FOUND. That encodes the ethical rule that silence is not safety.

7.5 Phase 3 test cases (case-level diligence)

ID	Focus	Expected
TC-21	Party names from executed-by deed	Names extracted
TC-22	Kannada sale-deed name extraction	Fallback still works
TC-23	Incomplete ownership evidence	Snippet attached
TC-24	Indian sale-deed name patterns	Seller/buyer found
TC-25	Survey prefix normalization	Sy. No. matches Survey No.
TC-26	Multiple documents one case	Shared case id
TC-27	Equal survey numbers	MATCH
TC-28	Different survey numbers	MISMATCH HIGH
TC-29	Different areas	MISMATCH
TC-30	Recalculate uses all files	Merged findings
TC-31	EC nil encumbrance	Mortgage NO_ISSUE_FOUND
TC-32	Missing evidence	Stays NOT_VERIFIED
TC-33	Coverage percent	Formula applied
TC-34	Risk score deterministic	DETECTED points only
TC-35	Finalize hash and ledger	GENESIS then record_hash
TC-36	Second report previous_hash	Links to prior record
TC-37	Untouched report verify	VALID
TC-38	Modified report verify	TAMPERED
TC-39	Unauthorized report access	403

7.6 Manual test plan

1. Landing -> Get Started -> sign up -> login -> dashboard.
2. Choose Residential -> Independent House -> create case.
3. Upload a PDF; confirm it appears as stored, not analyzed yet.
4. With SARVAM_API_KEY, Analyze a Kannada litigation sale deed. Expect Litigation DETECTED/HIGH; mortgage and approval NOT_VERIFIED.
5. Upload an EC that explicitly says nil encumbrance, Analyze, Recalculate. Mortgage may become NO_ISSUE_FOUND.
6. Upload a survey sketch with a different survey number. Expect MISMATCH HIGH.

7. Finalize. Confirm warning that unverified checks remain. Download PDF. Click Verify Integrity: VALID.
8. Attempt another user's URL: 403. Open /health without auth: ok.

7.7 Tools and re-run commands

From backend with the virtual environment active: `pytest -q`. To run one phase: `pytest tests/test_phase1.py -v`.
Frontend: `npm run build`. These commands do not call the live Sarvam API.

Chapter 8 Results

8.1 Delivered system

The implemented product is a working local application. A buyer can move from a public landing page to a hashed PDF without leaving the browser, provided Docker, the API, the Vite server, and (for live OCR) Sarvam are available. The royal-themed interface presents five checks in plain English: who owns it, court cases, bank loans, permissions, and land records.

8.2 Functional results

Area	Result observed
Authentication	Unique users, JWT session, protected routes
Residential gating	Non-residential create blocked; subscription UI shown
Upload safety	PDF/PNG/JPEG accepted; txt rejected
Kannada path	OCR language kn-IN; translate when Kannada detected
English path	Translation skipped when no Kannada letters
Litigation demo	Stay / O.S. language yields DETECTED HIGH
Absence rule	No EC keeps Mortgage NOT_VERIFIED
EC clearing	Explicit nil encumbrance can clear mortgage
Survey MATCH	Normalized prefixes compare equal
Survey MISMATCH	45/3 vs 45/4 becomes HIGH mismatch
Coverage vs score	Independent meters on due-diligence page
Integrity	Unchanged report VALID; edited JSON TAMPERED
Ownership	Cross-user access denied

8.3 Representative walkthrough

Upload SaleDeed_01_Litigation_Kannada.pdf and Analyze. The pipeline may classify the file as SALE_DEED or COURT_CASE_DOCUMENT if case numbers dominate keywords. Litigation is DETECTED at HIGH when stay, pending O.S. 234/2019, or transfer restriction appears. Mortgage and approval remain NOT_VERIFIED. Adding an EC with explicit nil encumbrance and recalculating can move mortgage to NO_ISSUE_FOUND. Adding a survey sketch with the same survey produces MATCH; a different survey produces MISMATCH. Finalize then Verify yields VALID for an untouched report.

8.4 Scoring example

A single DETECTED HIGH finding contributes 70 points and forces overall HIGH. Two DETECTED MEDIUM findings contribute 30 points and overall MEDIUM unless a HIGH is also present. A case that is entirely NOT_VERIFIED has score 0 and overall LOW, while coverage may still be 0% or 50% depending on whether related papers were uploaded. This result is deliberate: an empty vault is not a safe property; it is an incomplete file.

8.5 User-interface results

- Landing states Your Alvvocate. and lists the five checks.
- Vault lists files with Analyze actions and recommended papers.
- Analysis page shows pipeline steps, extracted groups, and evidence.
- Due diligence shows five category cards, comparison table, and unverified CTAs.
- Final report shows hash, ledger record, VALID/TAMPERED, and PDF download.

8.6 Test results

The repository contains 39 automated test functions. Phase 1 and Phase 2 were previously recorded as 20/20 passing in the Phase 1-2 test-case report. Phase 3 extends coverage to comparison, scoring, hashing, and tamper detection. Pytest does not bill Sarvam. A live Kannada PDF once failed when the OCR download was a ZIP; sarvam_service.py now unpacks ZIP payloads, which is a concrete result of testing against the real API.

8.7 Limitations observed

- Live OCR depends on scan quality and Sarvam uptime.
- A litigation-heavy sale deed may classify as Court Case Document.
- Boundary OCR lines can concatenate (north absorbing south/east/west).
- Kannada party names may fail extraction; incomplete ownership stays DETECTED, not cleared.
- OCR is requested as Kannada, so Hindi or Tamil pages are not first-class.
- The ledger is a local hash chain, not Ethereum or a public timestamp authority.
- Subscription and social login are UI demonstrations.
- Recalculate never re-OCRs; a bad first digitise must be re-run via Analyze.
- Deployment is local-student grade, not production-hardened.

8.8 Discussion

The main result is not a higher machine-learning F1 score. The main result is an honest interface: missing papers stay visible, evidence is quoted, and a hash can prove whether the JSON still matches what was finalized. Splitting Sarvam from Python made the viva story simple: AI reads, rules decide, tests freeze the rules. That is a stronger academic artefact than an untestable prompt that declares a title clear.

Chapter 9 Conclusion and Future Enhancement

9.1 Conclusion

This project designed and implemented BhoomiScan, an AI-assisted residential property due-diligence web application. It addresses a real buyer problem: scanned Kannada and English papers that hide ownership, litigation, mortgage, approval, and survey issues. The proposed hybrid method uses Sarvam Document AI only to digitise, translate, and extract, while a deterministic Python engine classifies, vetoes hidden litigation language, scores five categories with three honest states, compares documents, and seals a versioned report with SHA-256 and a GENESIS ledger.

Requirements for authentication, upload safety, analysis, comparison, and integrity were met in code and in automated tests. The system refuses to call an unverified property clear. It does not pretend to be a public blockchain, a payment product, or a government portal. Within those honest bounds, a student can demonstrate a complete path from signup to a VALID or TAMPERED report PDF.

9.2 Contributions

1. Three-state verification so missing papers remain NOT_VERIFIED.
2. Hybrid Indic OCR plus rules, with a source-text veto on extraction.
3. Case-level comparison with normalized survey prefixes and area tolerance.
4. Separation of coverage percent from risk score.
5. Tamper-evident local ledger with recomputed verification.
6. A 39-case pytest suite that mocks the paid AI boundary.

9.3 Future enhancement

- Named-entity recognition for Kannada party names, with user confirmation of document type.
- Optional support for additional Indian languages if the OCR provider exposes them.
- Government API integration where legally permitted, never scraping.
- Real payment processing only if the product is commercialized.
- RFC 3161 trusted timestamps to complement the local GENESIS chain.
- Mobile application and better handling of bound-register photographs.
- Human-in-the-loop review where an advocate can override a status with an audit note.
- Stronger production hardening: rate limits, object storage, secret rotation.

9.4 Closing remark

BhoomiScan is a first pass, not a final legal opinion. Used that way, it can help a buyer ask better questions before money changes hands. Used as a rubber stamp, it would be harmful. The software is written to make the first use natural and the second use difficult.

Chapter 10 References

10.1 Software and frameworks

1. FastAPI documentation. <https://fastapi.tiangolo.com/>
2. SQLAlchemy 2.0 documentation. <https://docs.sqlalchemy.org/>
3. PostgreSQL 16 documentation. <https://www.postgresql.org/docs/16/>
4. React documentation. <https://react.dev/>
5. Vite documentation. <https://vite.dev/>
6. Tailwind CSS documentation. <https://tailwindcss.com/docs>
7. Uvicorn. <https://www.uvicorn.org/>
8. Pydantic Settings. <https://docs.pydantic.dev/>
9. PyJWT. <https://pyjwt.readthedocs.io/>
10. fpdf2. <https://py-pdf.github.io/fpdf2/>
11. pytest. <https://docs.pytest.org/>
12. Docker Compose. <https://docs.docker.com/compose/>

10.2 AI and language technology

1. Sarvam AI, Document AI and translation APIs. <https://www.sarvam.ai/>
2. Unicode Consortium, Kannada block U+0C80 to U+0CFF.
3. National Institute of Standards and Technology, FIPS 180-4, Secure Hash Standard (SHA-256).

10.3 Domain and academic context

1. The Registration Act, 1908 (India), on registration of documents.
2. The Transfer of Property Act, 1882 (India), on sale and mortgages of immovable property.
3. Karnataka land records and registration practice (Bhoomi / Kaveri context) as publicly described by state departments. This project does not scrape those portals.
4. Pressman, R. S., Software Engineering: A Practitioner's Approach, McGraw-Hill.
5. Sommerville, I., Software Engineering, Pearson.
6. IEEE Recommended Practice for Software Requirements Specifications, IEEE Std 830.

10.4 Project artefacts

1. BhoomiScanV18 source repository: backend FastAPI application and frontend React SPA.
2. README.md, run instructions and Phase 1 journey.
3. docs/BhoomiScan_Test_Case_Report.md and generated Phase 1 / Phase 2 test PDFs.
4. backend/tests/test_phase1.py, test_phase2.py, test_phase3.py.
5. This report generated by docs/generate_detailed_project_report.py.

End of report. BhoomiScan - Your Alivocate. AI-assisted document review only. Not a legal guarantee.